

CAGOULE

v 1 . 5 . 0

Cryptographie Algébrique Géométrique par Ondes et Logique Entrelacée
[Rapport Technique Final](#)
Version stabilisée — 247 / 247 tests passés

Auteur	Slim Issa (slimissa)
Version	1.5.0
GitHub	github.com/slimissa/CAGOULE
PyPI	pypi.org/project/cagoule
Licence	MIT
Tests	247 / 247 ✓
Date	Avril 2026

Table des matières

1. Présentation du projet.....	3
1.1 Métriques de la version 1.5.0.....	3
1.2 Évolution v1.1 → v1.5.0	3
2. Architecture du pipeline	4
2.1 Vue d'ensemble	4
2.2 Pipeline de dérivation des paramètres	4
3. Analyse des modules	5
3.1 Nouvelles classes v1.5.0.....	5
4. Corrections et optimisations v1.5.0.....	6
4.1 Structure du projet — imports relatifs.....	6
4.2 Cache de la S-box — optimisation critique.....	6
4.3 Matrice de diffusion — déroulage de boucle	6
4.4 Paramètre params optionnel dans decrypt()	7
4.5 Correction des tests (mots de passe incohérents).....	7
4.6 Correction test_fp2 — diviseurs de zéro dans Fp^2	7
5. Primitives cryptographiques	8
5.1 Tableau récapitulatif	8
5.2 Format binaire CGL1	8
6. Fondements mathématiques CGS2025	9
6.1 Constantes CGS2025.....	9
6.2 Pilier Ω — fonction zêta et round keys	9
6.3 Stratégies pour μ (racine de x^4+x^2+1).....	9
7. Nouveautés v1.5.0.....	10
7.1 Nettoyage mémoire sécurisé — utils.py	10
7.2 Logging structuré — logger.py.....	10
7.3 Tests statistiques NIST SP 800-22	10
7.4 Benchmark — scripts/benchmark.py.....	11
8. Suite de tests.....	12
8.1 Résultat final.....	12
8.2 Répartition des tests par module	12
8.3 Known Answer Test (KAT) v1.5.....	12
9. État du dépôt et publication	13
9.1 GitHub — github.com/slimissa/CAGOULE	13
9.2 Dépendances Python	13
10. Conclusion.....	14
Récapitulatif des gains de performance	14
Points forts.....	14
Prochaines étapes — Phase 2	14

1. Présentation du projet

CAGOULE (Cryptographie Algébrique Géométrique par Ondes et Logique Entrelacée) est un système de chiffrement symétrique hybride conçu par Slim Issa. La version 1.5.0 constitue la version finale et stabilisée de la Phase 1 (implémentation Python), après une campagne de correction et d'optimisation documentée dans le CHANGELOG de maintenance.

Le projet fusionne des primitives cryptographiques modernes standardisées — ChaCha20-Poly1305 (RFC 8439), Argon2id, HKDF-SHA256 — avec des structures mathématiques avancées issues du Concours Général Sénégalais 2025 (CGS2025) : fonction zêta de Riemann, matrices de Vandermonde sur $\mathbb{Z}/p\mathbb{Z}$, extension quadratique $\mathbb{F}_{p^2}$, et S-boxes algébriques.

1.1 Métriques de la version 1.5.0

3 667 lignes de code	15 modules Python	12 classes définies	247 tests / 247 ✓
--	---	---	---

1.2 Évolution v1.1 → v1.5.0

La v1.5.0 corrige les problèmes structurels de la v1.1 et apporte des optimisations majeures de performance :

Dimension	v1.1 (initial)	v1.5.0 (final)
Structure	Imports absolus — ImportError	Imports relatifs — package correct
Déchiffrement 1Ko	~285 ms (KDF à chaque appel)	~11 ms (96% plus rapide)
S-box inverse	~79 ms / 10 000 appels	~2 ms (97% plus rapide)
Tests passés	~70%	100% — 247/247
Nettoyage mémoire	Absent	secure_zeroize() + context manager
Logging	Absent	DEBUG/INFO/WARNING/ERROR structuré
Tests NIST	Absent	14 tests statistiques SP 800-22
Benchmark	Absent	scripts/benchmark.py complet

2. Architecture du pipeline

2.1 Vue d'ensemble

CAGOULE chiffre en deux couches successives et complémentaires :

ENTRÉE → Plaintext (bytes ou str UTF-8)

↓ PKCS7 pad → blocs de 16 octets → éléments de $\mathbb{Z}/p\mathbb{Z}$

```

COUCHE 1 : CBC-like interne (algèbre  $\mathbb{Z}/p\mathbb{Z}$ )
Étape 1  v = mi + prev_cipher      mod p  (CBC mixing)
Étape 2  w = P × v                  mod p  (diffusion Vander.)
Étape 3  u = S-box(w) = w3 + cw ou wd mod p  (confusion)
Étape 4  c = u + round_key_Ω        mod p  (clé de ronde ζ(2n))
  
```

↓ T(message) sérialisé (8 octets / élément de $\mathbb{Z}/p\mathbb{Z}$)

```

COUCHE 2 : AEAD ChaCha20-Poly1305 (RFC 8439)
ChaCha20(K_stream, nonce) XOR T(message) → ciphertext
Poly1305 MAC sur AAD || ciphertext → tag 128 bits
AAD = Magic(4) || Version(1) || Salt(32)
  
```

SORTIE → CGL1: Magic(4) | Ver(1) | Salt(32) | Nonce(12) | CT | Tag(16)

2.2 Pipeline de dérivation des paramètres

Tous les paramètres cryptographiques sont dérivés déterministement depuis (password, salt) :

1. password + salt → Argon2id (t=3, m=64 MB) → K_master 512 bits
2. K_master → HKDF('CAGOULE_N') → n ∈ [4, 65536] (paramètre secret de $\mathbb{Z}(2n)$)
3. K_master → HKDF('CAGOULE_P') → p_seed → nextprime(p_seed | 2⁶³) ≈ 2⁶⁴
4. p → generate_mu(p) → μ racine de x⁴+x²+1 (Stratégie A dans $\mathbb{Z}/p\mathbb{Z}$ ou C dans $\mathbb{F}_p$)
5. K_master → HKDF('CAGOULE_DELTA') → δ → SBox.from_delta(δ, p)
6. μ + K_master → 16 nœuds distincts → DiffusionMatrix Vandermonde 16×16 mod p
7. K_master → HKDF('CAGOULE_ENC') → K_stream 256 bits (clé ChaCha20)
8. n + salt + p → $\mathbb{Z}(2n)$ → coefficients Fourier → HKDF → 64 round keys mod p

Entropie supplémentaire : n est secret (16 bits d'entropie sur [4, 65536]). Sans n, les round keys sont mathématiquement irrécupérables, même en connaissant K_master.

3. Analyse des modules

Le projet v1.5.0 comprend 15 modules pour 3 667 lignes de code, organisés en 6 couches fonctionnelles.

Fichier	Couche	Responsabilité	Lignes	Classes
cipher.py	Core	Chiffrement CBC-like + enveloppe AEAD ChaCha20-Poly1305	247	0
decipher.py	Core	Déchiffrement inverse + 3 exceptions typées (v1.5 : +params opt.)	273	3
params.py	Core	Dérivation complète + zeroize() + context manager __enter__	341	1
omega.py	Math	Pilier Ω : $\zeta(2n)$, coefficients Fourier, 64 round keys HKDF	228	1
sbox.py	Math	S-box $x^3+cx + \text{fallback } x^d$ — cache _FALLBACK_CACHE (v1.5)	329	1
matrix.py	Math	Vandermonde/Cauchy mod p + Gauss-Jordan + déroulage n=16 (v1.5)	339	2
mu.py	Math	Racine de x^4+x^2+1 — Stratégie A ($\mathbb{Z}/p\mathbb{Z}$) et C ($\mathbb{F}_p^2$)	302	1
fp2.py	Math	Extension quadratique $\mathbb{F}_p^2 = \mathbb{Z}/p\mathbb{Z}[t]/(t^2+t+1)$ — arithmétique	291	1
constants.py	Math	Constantes CGS2025 : ρ , β , $\zeta(8)=\pi^8/9450$, $x_0=3\pi$	152	0
format.py	Format	Parsing / sérialisation / inspection du format binaire CGL1	346	2
cli.py	CLI	Interface ligne de commande — 5 commandes, pipes UNIX (v1.5)	399	0
logger.py	Support	Logging structuré DEBUG/INFO/WARNING/ERROR — NOUVEAU v1.5	59	0
utils.py	Support	secure_zeroize(), SensitiveBuffer, analyze_sbox() — NOUVEAU v1.5	264	1
__init__.py	Package	API publique du package cagoule	90	0
__version__.py	Package	Métadonnées de version	7	0
TOTAL	15 mod.	—	3 667	12

3.1 Nouvelles classes v1.5.0

Classe	Module	Rôle
SensitiveBuffer	utils.py	Context manager : alloue un bytearray et l'efface automatiquement à la sortie du bloc with
OmegaInfo	omega.py	Rapport de diagnostic sur le pilier Ω pour un n donné (valeurs, vérification CGS2025)

4. Corrections et optimisations v1.5.0

Cette section documente dans le détail chaque correction apportée entre la v1.1 et la v1.5.0, avec le problème original, la solution implémentée et l'impact mesuré.

4.1 Structure du projet — imports relatifs

Problème

La v1.1 utilisait des imports absolus dans tous les modules internes (from cipher import encrypt, from params import ...). Lorsque CAGOULE était installé comme package pip, Python ne trouvait pas les modules frères et levait systématiquement ImportError.

```
$ python3 -m cagoule.cli encrypt test.txt -p 'mdp'
ImportError: attempted relative import with no known parent package
```

Solution

Remplacement systématique de tous les imports absolus par des imports relatifs dans 6 fichiers :

- from cipher import → from .cipher import
- from params import → from .params import
- from omega import → from .omega import (y compris l'import inline dans params.py)
- from fp2 import → from .fp2 import (dans mu.py — cas le plus oublié)

4.2 Cache de la S-box — optimisation critique

Problème

Dans la v1.1, la méthode `sbox_fallback_inverse()` recalculait `pow(d, -1, p-1)` à chaque appel. Pour $p \approx 2^{64}$, cette exponentiation modulaire géante était exécutée des milliers de fois par déchiffrement.

Solution — cache_FALLBACK_CACHE

Avant (v1.1)	Après (v1.5)
<pre>def sbox_fallback_inverse(y, p, d): # Recalculé à CHAQUE appel ! d_inv = pow(d, -1, p - 1) return pow(y, d_inv, p)</pre>	<pre>FALLBACK_CACHE = {} def _compute_fallback_params(p): if p in FALLBACK_CACHE: return FALLBACK_CACHE[p] d_inv = pow(d, -1, p-1) FALLBACK_CACHE[p] = (d, d_inv) return d, d_inv</pre>

Impact mesuré

Métrique	v1.1	v1.5
10 000 appels inverse()	~79 ms	~2 ms (×40)

4.3 Matrice de diffusion — déroulage de boucle

Problème

La multiplication matrice-vecteur $16 \times 16 \bmod p$ était réalisée par une double boucle Python sur le chemin critique (chaque bloc de 16 éléments). L'overhead de l'interpréteur Python pour 256 multiplications × nombre de blocs représentait une part significative du temps total.

Solution — matrix_vec_mul_mod_optimized

Ajout d'un chemin optimisé pour $n=16$ (taille fixe de CAGOULE) qui exprime les 256 multiplications directement, sans boucle intermédiaire :

```
def matrix_vec_mul_mod_optimized(m, v, p):
    n = len(m)
    if n == 16: # Chemin critique — CAGOULE utilise toujours n=16
        return [
            (m[0][0]*v[0] + m[0][1]*v[1] + ... + m[0][15]*v[15]) % p,
            (m[1][0]*v[0] + m[1][1]*v[1] + ... + m[1][15]*v[15]) % p,
            ... # 16 lignes, 0 boucle
        ]
    else:
        return [sum(m[i][j]*v[j] for j in range(n)) % p for i in range(n)]
```

Impact

Métrique	v1.1	v1.5
Déchiffrement 1 Ko (sans KDF)	~18.5 ms	~11 ms (-40%)

4.4 Paramètre params optionnel dans decrypt()

Problème

Dans la v1.1, chaque appel à `decrypt()` re-dérivait les paramètres depuis zéro via `Argon2id` (~285 ms par appel). Pour une application comme `cagoule-pass` qui déchiffre plusieurs fois avec le même mot de passe, ce coût était multiplicatif.

Solution

```
def decrypt(ciphertext: bytes, password: bytes | str,
            fast_mode: bool = False,
            params: CagouleParams | None = None) -> bytes:
    salt, nonce, ct_tag = _parse_cgll(ciphertext)
    if params is None:
        # Dérivation KDF si pas de params fournis (~285 ms)
        params = CagouleParams.derive(password, salt, fast_mode=fast_mode)
    # Sinon : réutilisation directe (~0 ms)
    ...
```

Impact

Métrique	v1.1	v1.5
Déchiffrement 1 Ko (avec KDF)	~285 ms	~11 ms (-96%)

4.5 Correction des tests (mots de passe incohérents)

Dans la v1.1, les tests utilisaient des mots de passe différents (`b"test_password"`, `b"pwd"`, `b"mon_mdp"`) alors que les paramètres `fast_params` étaient dérivés avec `b"test_password_cagoule"`. Résultat : `CagouleAuthError` systématique car `K_stream` ne correspondait pas. Correction : `PASSWORD_TEST = b"password_cagoule_2026"` partout, avec `fast_mode=True` cohérent entre `encrypt()` et `decrypt()`.

4.6 Correction test_fp2 — diviseurs de zéro dans F_p^2

Pour $p \equiv 1 \pmod 3$ (exemples : 7, 13, 19...), certains éléments non nuls de F_p^2 ont une norme nulle ($a^2 - ab + b^2 \equiv 0 \pmod p$). Ces éléments sont des diviseurs de zéro et ne sont pas inversibles — tenter de les inverser levait `ZeroDivisionError`, faisant échouer les tests. Correction : ajout du filtre `if (a^2-ab+b^2) % p == 0`: continue dans la boucle exhaustive.

5. Primitives cryptographiques

5.1 Tableau récapitulatif

Primitive	Algorithme	Paramètres et références
KDF principal	Argon2id	$t=3$, $m=64$ MB, $p=1$ — résistant GPU/ASIC
KDF fallback	Scrypt	$n=2^{17}$, $r=8$, $p=1$ — si argon2-cffi absent
AEAD	ChaCha20-Poly1305	RFC 8439 — nonce 96 bits aléatoire
Dérivation clés	HKDF-SHA256	RFC 5869 — contextes distincts par paramètre
Chiffrement interne	CBC-like sur $\mathbb{Z}/p\mathbb{Z}$	Blocs de 16 éléments — $p \approx 2^{64}$
Diffusion	Vandermonde 16×16	Inversible mod p , fallback Cauchy si collision
Confusion	x^3+cx (ou x^d)	Bijective — inversion Newton ou cache d_{inv}
Clés de ronde	$\mathbb{Z}(2n) \rightarrow \text{Fourier} \rightarrow \text{HKDF}$	64 clés, $n \in [4, 65536]$ secret
Primalité	Miller-Rabin	Déterministe pour $n < 3.3 \times 10^{24}$
Racine μ	Tonelli-Shanks / $\mathbb{F}_p^2$	$x^4+x^2+1=0$ — Stratégie A ou C automatique

5.2 Format binaire CGL1

Chaque message chiffré est sérialisé dans le format propriétaire CGL1. L'overhead fixe est de 65 octets.

Offset	Taille	Champ	Description
0 – 3	4 oct.	Magic	0x43474C31 = «CGL1»
4	1 oct.	Version	0x01
5 – 36	32 oct.	Salt	Sel Argon2id (inclus dans AAD Poly1305)
37 – 48	12 oct.	Nonce	Nonce ChaCha20 (96 bits, aléatoire)
49 – N	variable	CT	T(message) chiffré par ChaCha20
N – N+16	16 oct.	Tag	Tag Poly1305 — intégrité garantie
Overhead total	65 octets = 4 + 1 + 32 + 12 + 16		

6. Fondements mathématiques CGS2025

CAGOULE intègre les résultats du Concours Général Sénégalais 2025 comme source de constantes cryptographiques. Toutes les valeurs sont calculées avec mpmath à 60 chiffres décimaux significatifs, garantissant une précision bien au-delà des besoins cryptographiques.

6.1 Constantes CGS2025

Constante	Formule	Valeur numérique (60 DPS)
ρ — nombre d'or	$(1 + \sqrt{5}) / 2$	1.61803398874989484820458683436563811772...
β — volume de S	$(8\pi/81)(55\rho + 34)$	34.21... (positif, vérifié)
$\Omega = \zeta(8)$	$\pi^8 / 9450$	1.00407735619794433937868523850865172...
x_0	3π	9.42477796076937971538793784317798...
$\delta = (Z/13Z)^* $	$12 = p - 1$ pour $p=13$	12 — ordre du groupe multiplicatif de $Z/13Z$

6.2 Pilier Ω — fonction zêta et round keys

L'identité clé $\zeta(8) = \pi^8/9450$ est vérifiée à 60 décimales dans constants.py. Elle est exploitée pour générer les 64 clés de ronde selon le pipeline :

- Calculer $\zeta(2n)$ pour n dérivé du mot de passe ($n \in [4, 65536]$)
- Calculer les 64 coefficients : $a_k = (2/\pi) \times (-1)^{(k+1)} / k^{(2n)}$
- Pour chaque a_k : $\text{seed} = \lfloor |a_k| \times 2^{32} \rfloor \rightarrow \text{HKDF-SHA256}(\text{seed} || \text{salt} || n) \bmod p$

L'entropie de n (16 bits) rend les round keys irrécupérables sans ce paramètre, même en connaissant K_{master} .

6.3 Stratégies pour μ (racine de x^4+x^2+1)

$x^4+x^2+1 = (x^2+x+1)(x^2-x+1)$. Le système essaie deux stratégies dans cet ordre, garantissant que μ est toujours disponible :

Stratégie	Condition	Méthode
A	$p \equiv 1 \bmod 3 \rightarrow$ racine dans Z/pZ	Discriminant + Tonelli-Shanks sur $x^2 \pm x + 1 = 0$
C	$p \equiv 2 \bmod 3 \rightarrow$ pas de racine dans Z/pZ	$\mu = t$ dans $\mathbb{F}_{p^2} = Z/pZ[t]/(t^2+t+1)$ $t^3=1$ et $t^4+t^2+1 = t+(-t-1)+1 = 0 \checkmark$

7. Nouveautés v1.5.0

7.1 Nettoyage mémoire sécurisé — utils.py

Module entièrement nouveau. Fournit des primitives de zéroïsation pour les données sensibles.

Fonction / Classe	Description
<code>secure_zeroize(data)</code>	Écrase un bytearray avec <code>\x00 + ctypes.memset</code> — double protection contre le dead store elimination du compilateur
<code>bytes_to_zeroizable(d)</code>	Convertit bytes → bytearray pour permettre la zéroïsation ultérieure
<code>SensitiveBuffer(size)</code>	Context manager : alloue, permet l'usage, puis zéroïse automatiquement à la sortie du bloc with
<code>analyze_sbox(sbox, p)</code>	Analyse cryptographique exhaustive : uniformité différentielle δ , non-linéarité, distance aux affines
<code>sbox_report(analysis)</code>	Formatage texte lisible des résultats d' <code>analyze_sbox()</code>

Intégration dans `CagouleParams` (`params.py`) :

- Méthode `.zeroize()` : efface `K_master`, `K_stream`, `round_keys`, et met les références à `None`
- Support du context manager : `with CagouleParams.derive(...) as params: ...` — zéroïse automatiquement

7.2 Logging structuré — logger.py

Module entièrement nouveau. Logging centré sur le logger 'cagoule', configurable via la variable d'environnement `CAGOULE_LOG_LEVEL` ou `--verbose` / `--debug` dans la CLI.

Niveau	Déclencheur	Exemples d'événements loggés
DEBUG	<code>CAGOULE_LOG_LEVEL=DEBUG</code>	<code>K_master</code> dérivé (64 octets), <code>p = ...</code> , <code>n_zeta = ...</code>
INFO	<code>--verbose</code>	μ — stratégie A (dans <code>Fp2=False</code>)
WARNING	Automatique	S-box cubique : aucun c valide → fallback x^d
ERROR	Exceptions	Authentification échouée, format invalide

7.3 Tests statistiques NIST SP 800-22

14 tests statistiques implémentés en Python pur (zéro dépendance externe) sur 128 Ko de données chiffrées, validant que la sortie est indistinguable d'un aléatoire uniforme.

N°	Test	Résultat	p-value (exemple)
1	Frequency (Monobit) — équilibre 0/1	✓PASS	0.646
2	Block Frequency (M=128) — blocs uniformes	✓PASS	0.966
3	Runs — séquences consécutives	✓PASS	0.659
4	Longest Run — plus longue séquence de 1	✓PASS	> 0.01
5	Binary Matrix Rank — corrélation linéaire	✓PASS	> 0.01
6	DFT — périodicité spectrale	✓PASS	> 0.01
7	Approximate Entropy (m=5)	✓PASS	0.654

N°	Test	Résultat	p-value (exemple)
8a	Cumulative Sums (Forward)	✓PASS	0.469
8b	Cumulative Sums (Backward)	✓PASS	> 0.01
9	Serial (m=3) — uniformité des paires	✓PASS	0.499
10	Linear Complexity (M=500)	✓PASS	0.014
11	Non-overlapping Template — patterns fixes	✓PASS	> 0.01
12	Overlapping Template — séquences de 1	✓PASS	> 0.01
13	Universal Statistical — compressibilité	✓PASS	> 0.01
14	Random Excursions Variant	✓PASS	> 0.01

Seuil de signification $\alpha = 0.01$ — tous les tests dépassent ce seuil.

7.4 Benchmark — scripts/benchmark.py

Script complet de mesure des performances, avec 5 sections distinctes :

- Section 1 : KDF (Argon2id / Scrypt fast_mode) — durée en ms
- Section 2 : Chiffrement / Déchiffrement par taille (1Ko, 10Ko, 100Ko, 1Mo)
- Section 3 : Effet avalanche — % bits modifiés lors d'un changement de 1 bit en entrée
- Section 4 : Analyse S-box — uniformité différentielle δ , biais linéaire
- Section 5 : Usage mémoire via tracemalloc

Métrique	1 Ko	10 Ko	100 Ko
Chiffrement	~4 ms	~44 ms	~385 ms
Déchiffrement (sans KDF)	~11 ms	~77 ms	~450 ms
Effet avalanche	~50%	~50%	~50%

8. Suite de tests

8.1 Résultat final

247 / 247 ✓

run_tests.py — 1.1 secondes — 0 échec

8.2 Répartition des tests par module

Fichier de test	Tests	Ce qui est couvert
test_format.py	14	Magic, parse, serialize, inspect, is_cgl1, erreurs de format
test_sbox.py	22	Bijektivité exhaustive, roundtrip fallback, interface SBox, cache
test_matrix.py	17	verify_inverse, apply/apply_inverse roundtrip, fallback Cauchy
test_fp2.py	43	Arithmétique complète, relation $t^2+t+1=0$, inversion (avec filtre diviseurs de zéro), sqrt, conversions
test_mu.py	52	Stratégie A ($p \equiv 1 \pmod 3$) et C ($p \equiv 2 \pmod 3$), vérification algébrique, nœuds Vandermonde
test_cipher.py	35	Round-trips de 1 à 512 octets, auth error, altération, UTF-8, binaire
test_nist.py	15	14 tests NIST SP 800-22 + test T(message) couche algébrique
test_kat.py	12	Vecteurs KAT v1.5 : n_zeta, p, μ , K_stream, round_keys, CGL1 SHA-256
run_tests.py	247	Runner sans pytest — tous les tests consolidés

8.3 Known Answer Test (KAT) v1.5

Le vecteur KAT a été mis à jour vers la v1.5 avec les nouveaux paramètres dérivés :

Paramètre	Valeur
Version KAT	1.5
Mot de passe	CAGOULE_KAT_2026_MASTER_FIXED
Plaintext	"Hello, World!" (14 octets)
n_zeta dérivé	22 190
p dérivé	16 984 848 376 641 921 697 ($\approx 2^{63} \cdot 9$, premier)
Stratégie μ	A — racine trouvée dans $\mathbb{Z}/p\mathbb{Z}$
Taille CGL1	193 octets = 65 overhead + 128 ciphertext
SHA-256 CGL1	93cdda21c449...

9. État du dépôt et publication

9.1 GitHub — github.com/slimissa/CAGOULE

Indicateur	État
URL	https://github.com/slimissa/CAGOULE
Branche	master
Tests CI	GitHub Actions — .github/workflows/tests.yml
Badge tests	162 / 162 ✓ (base — run_tests.py : 247/247)
Licence	MIT
Langages	Python 99.1% · C 0.2% · CSS 0.5% · JS 0.1%
Dépôt PyPI	https://pypi.org/project/cagoule — v1.2.0 / v1.2.1 publiées

9.2 Dépendances Python

Paquet	Version min	Rôle	Type
cryptography	≥ 42.0	ChaCha20-Poly1305, HKDF-SHA256, Scrypt	Prod.
mpmath	≥ 1.3	Précision arbitraire — $\zeta(2n)$, constantes CGS2025	Prod.
argon2-cffi	≥ 23.0	KDF Argon2id — fallback Scrypt si absent	Prod.
pytest	≥ 7.0	Suite de tests avec fixtures	Dev
pytest-cov	≥ 4.0	Couverture de code	Dev

10. Conclusion

CAGOULE v1.5.0 est la version finale et stabilisée de la Phase 1 Python. Elle répond à l'ensemble des objectifs fixés : correction des bugs structurels, optimisations de performance mesurées, outillage de sécurité (zeroize), validation statistique (NIST), et suite de tests complète à 100%.

Récapitulatif des gains de performance

Métrique	v1.1	v1.5.0	Gain
Déchiffrement 1 Ko (avec KDF re-dérivé)	~285 ms	~11 ms	-96%
Déchiffrement 1 Ko (sans KDF)	~18.5 ms	~11 ms	-40%
S-box inverse (10 000 appels)	~79 ms	~2 ms	-97%
Tests passés	~70%	100%	+30 pts
Imports absolus (bugs packaging)	7 fichiers	0	-100%

Points forts

- Double couche de sécurité : algèbre originale $\mathbb{Z}/p\mathbb{Z}$ + ChaCha20-Poly1305 standardisé
- Corps de travail dynamique : $p \approx 2^{64}$ dérivé du mot de passe → paramètres uniques par session
- Entropie supplémentaire : n secret (16 bits) rend les round keys irrécupérables sans lui
- Robustesse par construction : fallbacks automatiques S-box (x^d), matrice (Cauchy), μ ($\mathbb{F}_p^2$)
- Zéroisation mémoire : K_master , K_stream , $round_keys$ effacés après usage
- Validation NIST : 14 tests SP 800-22 passés, p -values toutes > 0.01
- 247/247 tests passés — couverture $fp2$, μ (Stratégies A et C), KAT, NIST, cipher, format

Prochaines étapes — Phase 2

- cagoule-pass v1.1 — intégration du paramètre `params` dans `decrypt()`, auto-effacement presse-papier
- CAGOULE v2.0 — portage C : `libcagoule.so` avec `matrix.c` (`uint128_t`), `sbox.c`, bindings Python ctypes
- Vecteurs KAT cross-implémentation — valider C et Python contre les mêmes vecteurs

⚠ Avertissement de sécurité. CAGOULE est conçu à des fins académiques et de recherche. La couche algébrique interne (S-box x^3+cx , matrices Vandermonde sur $\mathbb{Z}/p\mathbb{Z}$) n'a pas subi d'audit de sécurité indépendant. En particulier, les propriétés de résistance différentielle et linéaire de la S-box ne sont pas garanties au niveau des standards industriels (AES). Ne pas utiliser en production sans audit cryptographique préalable.